

🎓 Student Information System

📌 Project Overview

The Student Information System is a simple Java-based application used to manage student records through the terminal.

The system allows users to add, view, search, update, and delete student information. Student records are stored in a file so that the data can be loaded again when the program is started.

This project is developed using Core Java concepts and does not require any graphical user interface or external libraries.

✨ Features

+ Add new student records 📋 Display all registered students 🔍 Search students using Student ID ✎ Update student information 🗑 Delete student records 📊 GPA validation between 0.0 and 4.0 🚫 Prevent duplicate Student IDs ⚠ Handle invalid user input 💾 Save student records to a file 📁 Load previously saved records 💻 Simple terminal-based menu

👤 Student Information Stored

The system stores the following information:

🆔 Student ID 👤 Student Name 🏠 Department 📈 GPA

🔧 Technologies Used

☕ Java 📦 Java Collections Framework 📁 Java File Handling 💾 Object Serialization 💻 Visual Studio Code 🐙 GitHub

📁 Project Structure

```
StudentInformationSystem/ | |—— StudentManagementSystem.java |—— students.dat
|—— README.md
```

📌 students.dat is automatically created by the program when student records are saved.

💻 Requirements Before running the project, make sure Java is installed on your computer.

Check Java:

```
java -version
```

Check the Java compiler:

```
javac -version
```

How to Run

1 Clone the Repository

Clone the repository from GitHub or download the project files.

2 Open the Project

Open the project folder in VS Code.

3 Open Terminal

Open:

Terminal → New Terminal

4 Compile the Program

Run:

```
javac StudentManagementSystem.java
```

5 Run the Program

Run:

```
java StudentManagementSystem
```

Main Menu

When the program starts, the following menu is displayed:

=====

**STUDENT INFORMATION SYSTEM Add
Student Display All Students Search Student
by ID Update Student Record Delete Student
Record Save & Exit**

The user can select an option by entering the corresponding number.

 Example  Adding a Student --- Add New Record --- Enter ID: 25BAI11478 Enter Full Name: Ayush Raj Singh Enter Department: Computer Science Enter GPA (0.0 - 4.0): 3.7

Student record added successfully.  Displaying Students --- Registered Students ---

```

+-----+-----+-----+-----+ | ID | Name | Department | GPA | +-----+
-+-----+-----+-----+ | 25BAI11478 | Ayush Raj Singh | Computer Science |
3.70 | +-----+-----+-----+-----+ |  Data Storage

```

The application uses Java Object Serialization to store student records.

The records are saved in:

students.dat

When the program starts, it checks whether the file exists. If previous records are available, they are loaded automatically. 

Input Validation

The program performs basic validation to avoid incorrect input.

For example:

 Duplicate Student IDs are rejected.  GPA must be between 0.0 and 4.0.  Invalid integer input is handled.  Invalid decimal input is handled.  Concepts Used

This project demonstrates several Java programming concepts:

 Classes and Objects  Constructors  Encapsulation  Private data members 
 Getters and Setters  ArrayList  Loops  Conditional Statements  Switch Statements
 Methods  Exception Handling  File Handling  Serialization  Deserialization 
 Scanner for User Input  Testing

The following operations can be tested after running the program:

 Add a new student  Try adding another student with the same ID  Display all students
 Search for an existing Student ID  Search for a non-existing Student ID  Update an existing student  Delete a student  Enter an invalid GPA  Enter invalid numerical input
 Exit and restart the program to verify that saved records are loaded  Limitations

This is a terminal-based application intended for academic/project use.

Currently, the system stores basic student information and does not include features such as:

 Login authentication  Database connectivity  Course registration  Advanced academic reports  Graphical user interface  Future Improvements

Possible future improvements include:

 Student login system  Database connectivity using MySQL  Attendance management
 Subject-wise marks  Semester-wise GPA  Course registration  Admin panel 
 Report generation  Author

